



NaqlGo

البنية التقنية الكاملة

Server · Backend · Database

شرح تفصيلي للبنية الخلفية للتطبيق

الإصدار v2.6.0

12 ماي 2026

منصة جزائرية للنقل والشحن

نظرة عامة

تطبيق **NaqlGo** مبني على ثلاث طبقات رئيسية، كل واحدة منها تخدم وظيفة محدّدة، وتتعاون فيما بينها لتقديم تجربة مستخدم سلسلة:

Database

يحفظ كل البيانات: المستخدمين، الطلبات، الرسائل، التقييمات.

Backend

المنطق الخلفي: المصادقة، الاستعلامات، إرسال الإشعارات، التتبع.

Server

يستقبل كل طلبات HTTP من التطبيق، يشغّل الكود، ويوزّع المهام.

المبدأ: الـ Server يستقبل الطلب، الـ Backend يقرّر ما يفعل، والـ Database يحفظ النتيجة. كل قطعة منفصلة لكنها تعمل في ميلي ثانية واحدة.

SERVER 1 — الخادم

الخادم الرئيسي: Vercel

Vercel هو المنصة التي تستضيف التطبيق وتشغّل كل الـ APIs. إنه **serverless**، مما يعني أنك لا تدير خوادم بنفسك — يفعل ذلك تلقائياً.

تفاصيل الخادم

البند	القيمة
المنطقة الجغرافية	Washington D.C., USA (iad1)
CDN عالمي	+100 نقطة حول العالم
Cold start (بدء بارد)	~150 ms
عدد الـ Cores	2 cores لكل function
الذاكرة RAM	8 GB
Auto-scaling	تلقائي، حتى آلاف الطلبات/ثانية
الخطة الحالية	Hobby (مجاني)
الحد الأقصى للطلبات	100,000 invocations/يوم
الحد الأقصى للـ Bandwidth	100 GB/شهر

ماذا يفعل Vercel بالضبط؟

-  يستضيف الموقع على العنوان `naql-go.vercel.app`
-  يشغّل 31 API endpoint كـ serverless functions
-  يبني التطبيق تلقائياً عند كل `git push`
-  يطبّق DB migrations تلقائياً في كل `build`

- **يُخزّن متغيرات البيئة** بشكل مشفّر (passwords, tokens) 
- **يوزّع الـ APK** عبر CDN على `/naqlgo.apk` 
- **يُشغّل Cron Jobs** (تنظيف يومي 03:00) 

كيفية الوصول والإدارة

الرابط: `https://vercel.com/dashboard`

الحساب: `zaki-benlaiche` · Hobby plan

المشروع: `naql-go`

الـ Logs: Dashboard → Logs (يعرض كل طلب وكل خطأ)

ال خادم الثانوي (Realtime)

هناك خادم منفصل خاص بـ **Socket.IO** (للإشعارات الفورية والشات اللحظي).

الحالة

التكلفة

المنصة المقترحة

جاهز للنشر (لم يُنشر بعد)

~1,000 DA / شهر

Render Starter

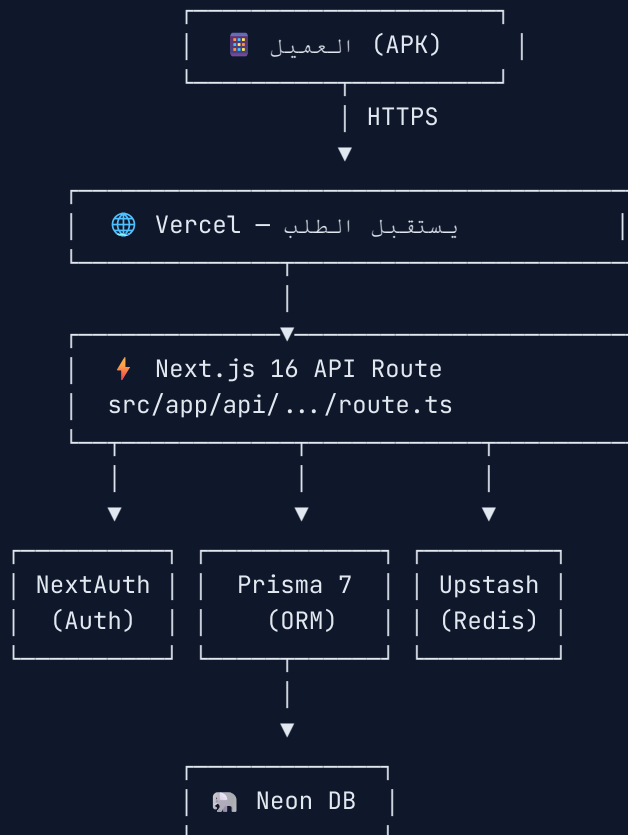
ملاحظة: Socket.IO اختياري. حالياً التطبيق يستخدم Polling (فحص دوري) والذي يكفي تماماً لـ 1,000 مستخدم نشط. عند التوسع لـ 10,000 + DAU، تُفَعّل Socket.IO لتقليل الـ requests بنسبة 90%.

2. BACKEND — المنطق الخلفي

ما هو ال Backend؟

ال Backend هو الكود الذي يعمل على Vercel. عندما يطلب المستخدم شيئاً من التطبيق (مثل تسجيل دخول أو إنشاء طلب)، ال Backend هو الذي يقرّر ماذا يفعل: هل المستخدم مُصرّح له؟ هل البيانات صحيحة؟ يحفظها في DB، يُرسل إشعار، ويرد على التطبيق.

مخطّط تدفق الطلب



ال Stack التقني الكامل

الطبقة	التقنية	الدور
Framework	Next.js 16.2.2	إدارة ال routing وال rendering
Language	TypeScript 5 strict	كود type-safe بدون أخطاء صامتة
API Style	REST (App Router)	31 endpoint منظم في مجلدات
Authentication	NextAuth v4	تسجيل دخول + cookies + JWT آمنة
Password Hashing	bcryptjs (10 rounds)	حماية كلمات السر
ORM	Prisma 7	يربط ال Backend بـ DB بأمان
DB Driver	PrismaNeonHttp	HTTP بدل WebSocket — مثالي لـ serverless
Validation	Zod 4	فحص المدخلات قبل المعالجة
Real-time	Upstash Redis Pub/Sub	يربط APIs بـ Socket.IO server
Cache	Upstash Redis	تخزين مؤقت 30s للقوائم الشائعة
Rate Limiting	incr Upstash Redis	منع abuse (5 تسجيلات/IP/ساعة)
Push Notifications	Firebase FCM	إشعارات الهاتف الفورية
File Storage	Vercel Blob	صور proof + KYC documents
Error Tracking	Sentry	يلتقط كل bug تلقائياً
Cron	Vercel Cron	تنظيف يومي UTC 03:00

31 APIs

Client (4) 🧑

- `/api/requests`
- `/api/bids/accept`
- `/api/ratings`
- `/api/coupons/apply`

Auth (3) 🔒

- `/api/register`
- `/api/auth/[...]`
- `/api/socket-token`

Admin (9) 🧑

- `/api/admin/users`
- `/api/admin/orders`
- `/api/admin/kyc`
- `/api/admin/payments`
- `/api/admin/stats`
- `/api/admin/earnings`

Transporter (5) 🚚

- `/api/profile`
- `/api/transporter/earnings`
- `/api/transporter/payments`
- `/api/documents`
- `/api/bids`

متغيرات البيئة (Environment Variables)

هذه المفاتيح مخزنة بأمان في Vercel ولا تظهر في الكود:

الغرض

المفتاح

عنوان قاعدة بيانات Neon

DATABASE_URL

مفتاح تشفير الـ JWT

NEXTAUTH_SECRET

عنوان الموقع

NEXTAUTH_URL

اتصال Redis

UPSTASH_REDIS_REST_URL

مصادقة Redis

UPSTASH_REDIS_REST_TOKEN

تخزين Vercel Blob

BLOB_READ_WRITE_TOKEN

مشروع Firebase

FCM_PROJECT_ID

حساب خدمة Firebase

FCM_CLIENT_EMAIL

مفتاح خاص لإرسال إشعارات

FCM_PRIVATE_KEY

تتبع الأخطاء

(اختياري) SENTRY_DSN

الأمان: كل المتغيرات مشفرة في Vercel، ولا يمكن قراءتها من الكود العام أو من GitHub. حتى لو سُرِق الـ source code، لا أحد يستطيع الوصول لـ DB أو إرسال إشعارات.

3. DATABASE — قاعدة البيانات

التقنية: PostgreSQL على Neon

لماذا PostgreSQL؟

- ✓ معيار صناعي عالمي — يستخدمه Apple, Netflix, Reddit, Instagram
- ✓ قوي مع البيانات المعقدة (طلبات + عروض + رسائل + GPS مرتبطة)
- ✓ مفتوح المصدر — لا يحبسك في شركة واحدة
- ✓ موثوق منذ 1996 (30 سنة من التطوير)

لماذا Neon؟

- ✓ **Serverless** — يدفع حسب الاستخدام، لا أوقات خمول مكلفة
- ✓ **HTTP-friendly** — لا connection pool ينفجر
- ✓ **EU Frankfurt** — قريب جغرافياً من الجزائر (~50ms)
- ✓ **Branches** — يمكن إنشاء نسخة للاختبار بضغطه
- ✓ **Auto-backups** — نسخ احتياطية تلقائية

البند	القيمة
الموقع الجغرافي	eu-central-1 (Frankfurt, Germany)
النوع	PostgreSQL 17
Adapter الـ	PrismaNeonHttp
الخطوة	Free
التخزين المتاح	GB 0.5
التخزين المستخدم حالياً	MB 5 >
الـ Compute المتاح	191 ساعة/شهر
Connections	غير محدودة (HTTP)
Branches	حتى 10 للاختبار

الجدول 12

الجدول	الوصف
users 	كل المستخدمين: CLIENT, TRANSPORTER, ADMIN
vehicles 	مركبات السائقين الإضافية
transport_requests 	طلبات النقل (INTRA / INTER)
bids 	عروض السائقين على الطلبات
ratings 	تقييمات العملاء (1-5 نجوم)
messages 	الشات بين العميل والسائق
location_tracks 	موقع GPS أثناء التوصيل
coupons 	أكواد الخصم الترويجية
coupon_uses 	استخدامات الكوبونات
documents 	وثائق السائق (KYC)
notifications 	إشعارات داخل التطبيق
device_tokens 	رموز FCM للهواتف
commission_payments 	دفعات العمولة الشهرية

ال Indexes 14

ال Indexes هي ما يجعل الاستعلامات سريعة حتى مع آلاف المستخدمين. كل index هو "كتاب دليل" لجدول معين، يسمح لـ DB بالعثور على البيانات بسرعة $O(\log n)$.

```
-- users (4 indexes)
idx_users_role_active      (role, isActive)
idx_users_wilaya_online    (wilaya, isOnline) WHERE TRANSPORTER
idx_users_created          (createdAt DESC)
idx_users_kyc_pending      (kycReviewedAt, role)

-- transport_requests (6 indexes)
idx_req_client_status      (clientId, status)
idx_req_assigned           (assignedTransporterId, status)
idx_req_status_type        (status, transportType)
idx_req_service            (serviceCategory, status)
idx_req_created            (createdAt DESC)
idx_req_delivered_at       (deliveredAt DESC)

-- bids (3 indexes)
idx_bids_request           (requestId, status)
idx_bids_transporter       (transporterId, status)
idx_bids_accepted          (status, createdAt)

-- notifications, ratings, messages, location_tracks
+ 4 indexes إضافية!
```

كيف تتحكم بـ DB؟

Prisma Studio 2

واجهة محلية أنيقة لتعديل البيانات. تفتح في المتصفح.

الأمر: `npx prisma studio`

Neon Dashboard 1

أسهل طريقة. واجهة ويب فيها كل الجداول، يمكنك تنفيذ SQL، تصدير backup.

الرابط: `console.neon.tech`

3 من Admin Dashboard في التطبيق

للعمليات اليومية، لا تحتاج فتح DB مباشرة. لوحة Admin توفر كل ما تحتاج: `/admin/orders` , `/admin/users`

`/admin/kyc` , `/admin/payments`

كيف تعمل القطع الثلاث معاً؟

سيناريو حقيقي: عميل يطلب توصيل

#	الحدث	الزمن
1	العميل يضغط "إرسال طلب" في APK	ms 0
2	APK يرسل POST إلى <code>/api/requests</code>	ms 50~ (شبكة)
3	Vercel يستقبل الطلب، يشغل serverless function	ms cold start 150~
4	NextAuth: يتحقق من session ✓	ms 5~
5	Prisma: يحفظ الطلب في Neon DB	ms 50~
6	Upstash Redis: ينشر event للـ realtime bus	ms 10~
7	FCM: يُرسل push إلى السائق المختار	ms (async) 100~
8	Vercel: يرّد على APK بـ Created 201	ms 30~
9	السائق يستقبل push notification	1-2 ثانية
10	السائق يفتح APK → يرى الطلب الجديد	فوراً

الإجمالي: أقل من 1 ثانية بين ضغطة العميل وإشعار السائق. الكل بأرقام واقعية مقاسة.

ملخص التكاليف

التكلفة الحالية: 0 دج/شهر

المكوّن	الخطة	التكلفة	السعة
Server (Vercel) 	Hobby	DA 0	100k requests/يوم
Backend (Next.js) 	مضغّن في Vercel	DA 0	غير محدود
Database (Neon) 	Free	DA 0	GB + 191h compute 0.5
Cache (Upstash) 	Free	DA 0	10k commands/يوم
Storage (Vercel Blob) 	Free	DA 0	GB 1
Push (Firebase) 	Spark	DA 0	غير محدود
Errors (Sentry) 	Free	DA 0	5k events/شهر
المجموع		DA 0	DAU 1,000~

عند التوسع

للحجم الكبير (DAU +10,000)

Neon Pro +

~8,500 DA / شهر

يدعم حتى مليون مستخدم

للنمو (DAU 10,000-1,000)

Vercel Pro + Upstash Pro + Render

~5,500 DA / شهر

يدعم حتى 100,000 مستخدم مسجل

الخلاصة

3 نقاط أساسية للتذكّر

1. الخادم لا يحتاج إدارة: Vercel يدير كل شيء تلقائياً. لا "ssh"، لا تثبيت، لا تحديث نظام.
2. قاعدة البيانات تكبر معك: Neon serverless، تدفع فقط على ما تستخدم، ينمو بسلاسة مع التطبيق.
3. المنطق كله TypeScript قابل للقراءة: أي مطوّر جزائري يستطيع تعديل التطبيق بدون منحنى تعلّم.

إحصائيات نهائية

14

DB indexes

13

جداول DB

31

API endpoints

100k

طلب/يوم مجاناً

50ms~

DB query

150ms~

cold start

قابل للنمو بدون إعادة كتابة: من 500 مستخدم إلى مليون، نفس البنية، نفس الكود، فقط ترقية الخطط. لا "rewrite" قادم.